Hito 2 – Documentación previa a la codificación

Javier Pérez Báez

1. Sobre trabajar tanto en la nube como en local

Para poder pasar de un modelo únicamente *online* o que éste pueda tener un modo *offline* con sincronización automática, habría que considerar:

- **Base de datos local**: se encarga de almacenar los datos cuando no hay conexión. En principio, se valora, como tecnología para la base de datos, **SQLite**, ya que es ligera y no requiere servidor. Evidentemente, la estructura de esta base de datos sería una copia idéntica de la base de datos remota de PostgreSQL
- **Detección de cambios**: saber qué se modificó durante la desconexión. Para ello, habría que añadir dos **campos**: un ***timestamp* de modificación** (para saber cuándo cambió cada registro) y un **estado de la sincronización** (para marcar registros pendientes). También sería necesaria una nueva **tabla** que represente los **cambios pendientes**, que funcione como una cola de cambios para registrar las operaciones *offline*. Además, se valora **HttpClient** como servicio de sincronización, que gestione la comunicación entre ambos estados.
- **Sincronización bidireccional**: al reconectar, fusionar cambios locales y remotos sin conflictos.

De esta manera, habría que prever tres posibles escenarios:

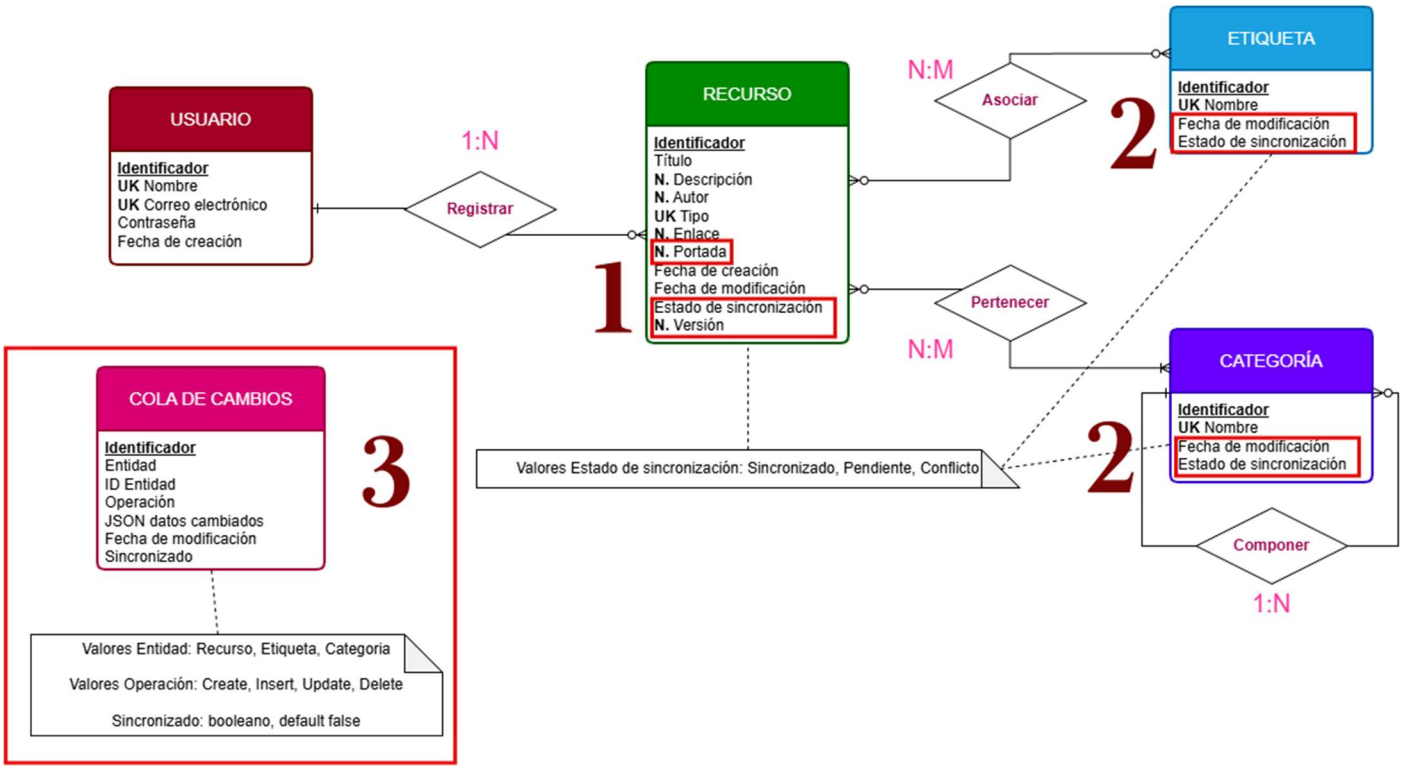
- Funcionamiento *online*:
    - Las operaciones van directamente a la API remota.
    - Se actualiza también la base de local (caché).
    - Estado de la sincronización: sincronizado.
- Funcionamiento *offline*:
    - La app detecta que no hay conexión a la API.
    - Las operaciones CRUD se guardan en base de datos local.
    - Se registra cada cambio en la tabla de cambios pendientes.
- Reestablecimiento de la conexión:
    - Un servicio en segundo plano detecta internet.
    - Se envían todos los cambios pendientes a la API.
    - Se reciben los cambios remotos que ocurrieron mientras se estaba *offline*.
    - Se fusionan los datos usando *timestamps*: prevalece el más reciente.
    - Se actualizan ambas bases de datos.

De cualquier forma, siendo esto un bosquejo y habiendo que adaptarlo al tiempo disponible para realizar el proyecto, se proponen tres fases de implementación de estos escenarios a cumplir en función de dicho tiempo:

- Fase 1: funcionamiento exclusivamente *online*, con una API Spring Boot y PostgreSQL en AWS.
- Fase 2: añadir una base de datos local (caché) para plantear un funcionamiento *offline* de sólo lectura.
- Fase 3: sincronización completa bidireccional con una cola de cambios *offline*.

Para finalizar, se dejan señalados algunos conceptos que puede ser interesante valorar a la hora de implementar este servicio: sincronización de bases de datos con replicación y *merge* bidireccional, control de concurrencia optimista usando versiones o *timestamps*, patrones de diseño para apps que funcionan sin conexión como *offline-first architecture*, y SQLite y su integración en JavaFX.

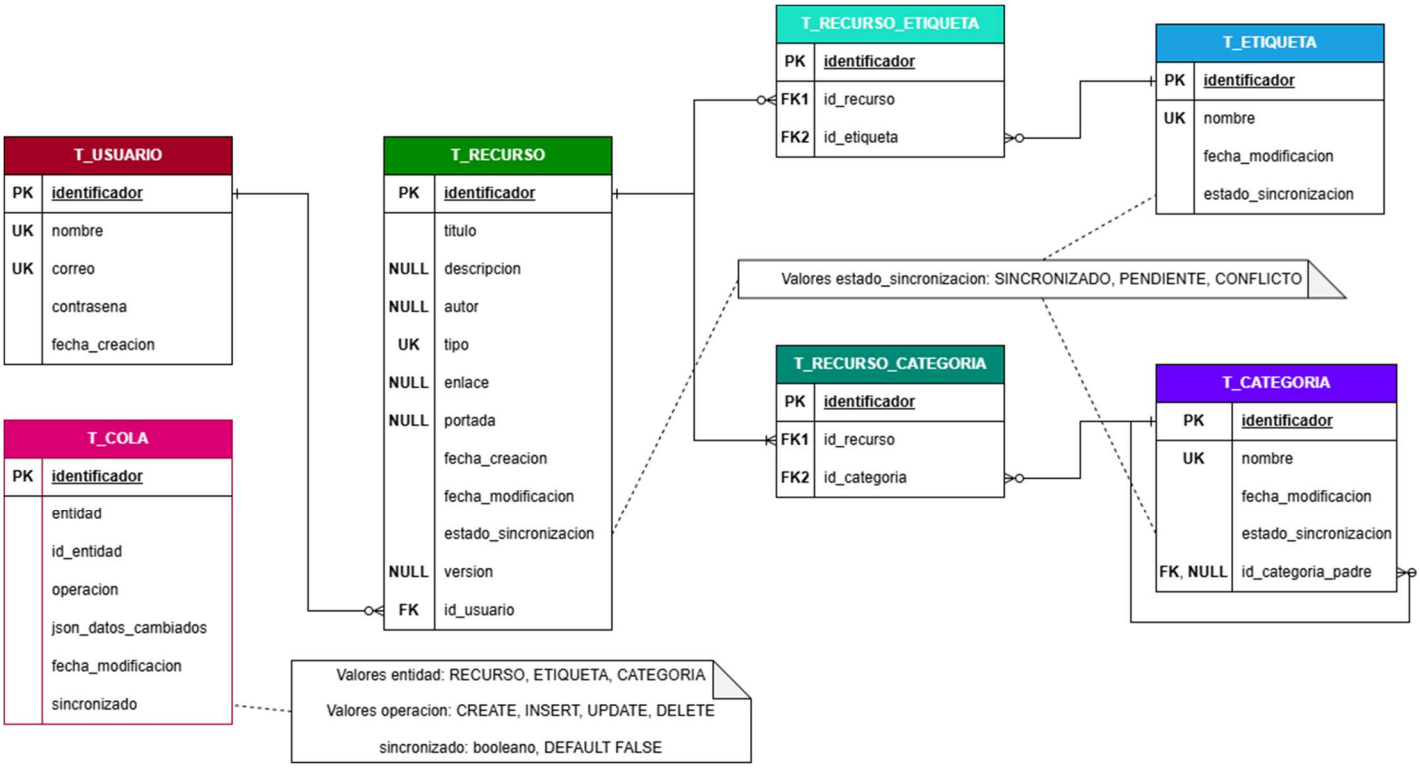
Por todo ello, el diagrama entidad/relación quedaría rediseñado con estos tres cambios:



- Cambio 1: la entidad Recurso ya disponía de un campo para la fecha de modificación, así que se añade un campo de tipo enumerado Estado de sincronización (el cual sólo permitirá los valores Sincronizado, Pendiente o Conflicto) y otro (que permite valores nulos) Versión de tipo entero. <u>También</u> se añade otro campo, Portada, que albergará una imagen y también permite valores nulos.
- Cambio 2: las entidades Etiqueta y Categoría añaden, amén del campo Estado de sincronización, el campo (*timestamp*) Fecha de modificación.
- Cambio 3: se añade una tabla independiente que funcione como una suerte de registro de auditoría, algo así como una cola de trabajo. La tabla contiene

un campo Identificador (autoincremental numérico), un campo Entidad (enumerado, que indica a qué entidad afecta el cambio), otro ID Entidad (numérico), otro JSON datos cambiados (de tipo texto, que guarda una captura del estado), campo (*timestamp*) Fecha de modificación y otro booleano, Sincronizado (con valor *false* por defecto).

Por lo tanto, el consecuente diagrama relacional quedaría de esta forma:



De cualquier forma, se adjunta a este documento un *script* SQL con el DDL de la base de datos para PostgreSQL.

2. Sobre trabajar con bases de datos documentales

Parece adecuado, para trabajar con documentos, utilizar una base de datos documental. En este sentido, se valora MongoDB como opción por el hecho de ser la única que conozco por haberla tratado en clase.

Trabajar con MongoDB, respecto a cualesquiera de las formas SQL, presenta ciertas ventajas, como el hecho de que su esquema es más flexible, permitiendo que toda la información resida en un solo documento BSON de manera anidada, sin necesidad de repartir la información en diferentes tablas. Este documento ya contendría todos los tipos de datos necesarios (metadatos, etiquetas, archivos adjuntos, etc.) y el acceso a éste sería más directo, en una sola consulta, puesto que ya no sería necesario recorrer varias tablas con *JOINS* para obtener información sobre un recurso.

La forma de integrar MongoDB en la arquitectura ideal sería confiar en ésta como la base de datos principal, sustituyendo a la de PostgreSQL. En este sentido, y en relación con el punto anterior, MongoDB funcionaría como la base de datos general *online*, mientras que se mantendría SQLite como base de datos *offline* por los motivos anteriormente expresados. Así todo, Spring Boot se conectaría a MongoDB mediante Spring Data MongoDB, se eliminaría la necesidad de JPA/Hibernate y las tablas relacionales, las entidades Java se conviertirían en @Document en lugar de @Entity y las relaciones se modelarían con documentos anidados o referencias.

De igual manera que con el punto anterior, se proponen tres fases de implementación de este aspecto en función del tiempo:

- Fase 1: mantener el modelo relacional actual con PostgreSQL para el diseño conceptual, pues así es más fácil de entender, documentar y presentar.
- Fase 2: probar MongoDB como base de datos principal, migrando una parte pequeña para experimentar y usando Spring Data MongoDB.
- Fase 3: implementar el modo *offline* con SQLite, creando un servicio de sincronización bidireccional entre SQLite y MongoDB.

Así pues, un ejemplo de una colección podría tener la siguiente estructura:

```
_id: ObjectId('65c3b9a8e4b0f1a2b3c4d5e6')
titulo : "Cien años de soledad"
descripcion : "Novela fundamental del realismo mágico"
autor : "Gabriel García Márquez"
tipo : "Libro"
enlace : "https://es.wikipedia.org/wiki/Cien_a%C3%B1os_de_soledad"
portada : "https://upload.wikimedia.org/wikipedia/commons/thumb/a/a1/Cien_a%C3%B1…"
fecha_creacion : 2026-02-15T10:30:00.000+00:00
fecha_modificacion : 2026-02-15T10:30:00.000+00:00
estado_sincronizacion : "SINCRONIZADO"
version : 1
id_usuario : 1
categoria : Object
  nombre : "Literatura"
  subcategoria : "Novela"
etiquetas : Array (3)
  0: "realismo mágico"
  1: "clásico"
  2: "colombiano"
```

3. Sobre la instancia S3 de Amazon Web Services

S3 significa *Simple Storage Service* (Servicio de Almacenamiento Simple).

Se puede entender como un almacenamiento que, en lugar de funcionar con carpetas, lo hace con "cubos" (*buckets*) en la nube de Amazon, donde se puede guardar cualquier tipo de archivo: imágenes, PDFs, vídeos, etc.

Algunos conceptos clave a tener en cuenta sobre Amazon S3:

- ***Bucket*/Cubo**: es la carpeta principal donde se guardan archivos. El nombre del *bucket* debe ser único en todo el mundo.
- **Objeto**: cada uno de los archivos a almacenar.
- **Clave/*Key***: es el nombre y la ruta del archivo dentro del *bucket*.
- **Región**: es el lugar físico del mundo donde se guardan los objetos. Elegir la región más cercana a los usuarios reduce la latencia.

Dado que la aplicación pretende gestionar colecciones digitales, parece bastante adecuado usar la instancia S3 de AWS para:

- **Almacenar los archivos**: la base de datos guarda la información textual (título, autor, descripción, etc.), pero los archivos pesados (PDFs, imágenes, etc.) se pueden guardar en S3 .
- **Generar URLs públicas o privadas**: se pueden obtener enlaces para compartir o descargar los archivos. Incluso se podrían generar URLs firmadas que caducan después de un tiempo (por ejemplo, un enlace de descarga válido por 24 horas) .
- **Copias de seguridad**: se puede usar S3 para hacer *backup* de la base de datos.
- **Escalabilidad automática**: si la aplicación crece, S3 puede manejar la cantidad de archivos.

Al parecer, la integración con Spring Boot es directa, mediante el AWS SDK para Java 2.x.

De cualquier modo, a pesar de que todo en S3 es privado y nadie puede acceder a los archivos sin permiso, sería conveniente tener en cuenta las siguientes buenas prácticas en lo relativo a seguridad:

- **No hacer los *buckets* públicos**: a menos que se quiera que cualquiera pueda ver, por ejemplo, las portadas de los libros.
- Usar **URLs prefirmadas** para compartir archivos temporalmente.
- Configurar el ***backend* con credenciales de IAM limitadas**, que sólo permitan acceder a S3 y a *buckets* específicos.